# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE CODE: CSA14** **COURSE NAME: Compiler Design**

**COURSE OUTCOME COVERED**

**CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)**

**Assessment Tool Weightage: 20%**

**QUESTIONS: 3** **Total Marks: 50**

**Annexure A**

## ERROR ANALYSIS TASK

### Code of Conduct:

I NISHA S - 192572083 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

NISHA S

## Q1. Analyze Lexical Recognition in a Simple Payroll Language

A payroll-oriented programming language uses identifiers, integer and floating-point constants, keywords, arithmetic operators, relational operators, assignment operators, and separators. Analyze how its lexical analyzer recognizes the following input:

```
float basic = 25000.50; int
bonus = 5000;
if (basic >= 20000 && bonus > 0) {
total = basic + bonus;
} print(total);
```

### (a) Token Specification

1. Define suitable regular expressions for identifiers, integer constants, floating-point constants, keywords, arithmetic operators, relational operators, assignment operators, and separators.
2. Identify the token categories that may overlap and explain how the lexical analyzer should resolve such overlap.
3. Explain why whitespace and line breaks do not normally form tokens in this language.

### (b) Lexical Recognition and Scanning

1. Trace the recognition of the operators >=, &&, and > in the given input. Explain the role of lookahead while recognizing multi-character operators.
2. Construct the token stream for the first three statements and explain how the scanner determines the boundary between each token.
3. Describe what should happen when an unrecognized character is encountered during scanning.

### (c) Longest-Match Analysis

1. Explain how the longest-match principle affects the recognition of >= and the floating-point constant 25000.50.
2. Consider a scanner in which the rule for an integer constant is placed before the rule for a floating-point constant. Analyze the possible tokenization of 25000.50 and explain how the specification can avoid the problem.
3. Explain how keyword recognition should be handled when a keyword such as print has the same character pattern as an identifier.

### (d) Extension and Evaluation

1. Modify the identifier rule so that an identifier may begin with an underscore. Give the revised regular expression and two valid examples.
2. Add the keyword payroll to the language. Explain how the scanner can distinguish payroll from identifiers such as payrollData and payroll_total.
3. Propose three test inputs that would help evaluate the correctness of the lexical analyzer, including at least one invalid input.

## Q2. Design and Evaluate a LEX Specification for a Sensor Monitoring Language

A small sensor-monitoring language is to be implemented using LEX/FLEX. It supports keywords, identifiers, numeric values, comparison operators, assignment, separators, string literals, and comments.

The language specification is:

- Keywords: sensor, read, display, if
- Identifiers: begin with a letter and may contain letters, digits, and underscores
- Numeric constants: integers or decimal values
- String literals: enclosed within double quotation marks
- Operators: +, -, *, /, <, <=, >, >=, ==, !=
- Assignment operator: =
- Separators: (, ), {, }, ;, ,
- Single-line comments begin with # and continue to the end of the line

- The scanner should ignore whitespace and comments while returning meaningful tokens to the parser.

```
sensor temperature; read
temperature; if (temperature >=
30) {     display "High
temperature";
}
```

### (a) Token and Pattern Design

1. Develop regular expressions for identifiers, integer and decimal constants, string literals, keywords, operators, assignment, and separators.
2. Explain how the scanner should distinguish a keyword such as sensor from an identifier such as sensorValue.
3. Explain why the rules for <= and >= must be designed carefully when single-character operators < and > are also present.

### (b) LEX Program Construction

1. Develop the declarations and rules sections of a LEX/FLEX specification for the language. The actions should display the token type and the recognized lexeme.
2. Include rules for ignoring whitespace and comments beginning with #.
3. Add a suitable rule for detecting an invalid or unterminated string literal and explain the diagnostic produced.

### (c) Token Stream Validation

1. Trace the given input through the LEX specification and write the expected sequence of tokens.
2. Identify the lexemes returned as keywords, identifiers, constants, operators, separators, and string literals.
3. Explain how the scanner handles the string literal "High temperature" as a single lexical unit.

### (d) Testing and Evaluation

1. Create three additional test inputs: one containing a decimal value, one containing a multi-character relational operator, and one containing an invalid character.
2. For each test input, state the expected token sequence or lexical error.
3. Evaluate whether the proposed test cases are sufficient to validate the major lexical rules and suggest one additional edge case.

## Q3. Trace a Source Expression from Lexical Analysis to Intermediate Code

Consider the following statement. Use it to analyze the information produced by lexical analysis and how that information is used by later compiler phases.

```
finalAmount = baseAmount + taxRate * subtotal;
```

### (a) Lexical Analysis

1. Identify every lexeme in the statement and classify it as an identifier, constant, operator, or separator.
2. Construct the complete token stream in the order produced by the lexical analyzer.
3. Explain how whitespace is handled and why the semicolon is significant to later phases even though it does not carry a numerical value.

### (b) Syntax and Symbol Information

1. Show the syntactic structure of the expression and indicate how operator precedence affects the grouping of + and *.
2. Identify the identifiers that would require symbol-table information and state examples of information that could be stored for them.
3. Explain how the syntax analyzer uses the token stream to determine the structure of the assignment statement.

### (c) Intermediate Representation Analysis

1. Construct a suitable three-address-code representation for the statement.
2. Explain why the multiplication is performed before the addition in the intermediate representation.

3. Identify the temporary variable introduced by the intermediate representation and explain its purpose.

**(d) Phase Dependency Analysis**

1. Explain what information is supplied by lexical analysis to syntax analysis and what additional information is obtained during semantic analysis.
2. Analyze what could happen if the lexical analyzer incorrectly classified * as an identifier.
3. Explain how an error in token recognition could affect syntax analysis and the generation of intermediate code.

**(e) Advanced Compiler Perspective**

1. Extend the language by introducing the operator ** for exponentiation. Explain how the lexical analyzer should distinguish ** from *.
2. Explain how line and column positions stored with tokens can improve error reporting during later compiler phases.
3. Propose two test statements that would verify correct recognition of *, **, +, and identifiers.

## RUBRICS FOR EVALUATION

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| **Error Identification** | 20% | Accurately identifies all errors with clear understanding of issues | Identifies most errors with minor omissions | Identifies limited errors; some are missed | Fails to identify key errors |
| **Root Cause Analysis** | 30% | Clearly explains underlying causes with strong technical reasoning | Explains causes with moderate clarity | Partial explanation; weak reasoning | Unable to identify causes of errors |
| **Correction & Justification** | 25% | Provides correct solutions with strong justification and logic | Provides correct corrections with some justification | Corrections are partially correct or weakly justified | Incorrect or no corrections provided |
| **Application of Concepts** | 15% | Effectively applies relevant concepts to analyze and resolve errors | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply concepts |
| **Clarity & Presentation** | 10% | Presents analysis clearly, logically, and systematically | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |

# ANSWERS

The answers below follow the question order and emphasize accurate error identification, root-cause analysis, justified corrections, application of compiler concepts, and clear presentation, as required by the evaluation rubric.

## Q1. Analyze Lexical Recognition in a Simple Payroll Language

### (a) Token Specification

**1. Suitable regular expressions**

```
Identifier: [A-Za-z][A-Za-z0-9_]*
Integer constant: [0-9]+
Floating-point constant: [0-9]+\.[0-9]+
Keywords: float|int|if|print
Arithmetic operators: [+-/*]
Relational operators: >=|<=|==|!=|>|<
Logical operator used in the input: &&
Assignment operator: =
Separators: [(){};,]
Whitespace to ignore: [ \t\r\n]+
```

The floating-point expression requires digits on both sides of the decimal point. If the language must also accept forms such as 25. or .50, separate patterns can be added deliberately.

**2. Overlapping token categories and resolution**

Keywords and identifiers overlap because words such as float, int, if, and print also match the identifier pattern. The scanner should first form the longest identifier-like lexeme, then classify an exact reserved word as a keyword, either by placing keyword rules before the identifier rule or by looking up the lexeme in a keyword table.

Integer and floating-point constants overlap because the prefix 25000 of 25000.50 matches the integer rule. Maximal munch selects the longer floating-point lexeme. Relational and assignment operators also share prefixes: > is a prefix of >=, < of <=, = of ==, and ! of !=. The scanner must look ahead and choose the longest valid operator. When two rules match the same longest text, rule priority decides the result.

**3. Whitespace and line breaks**

Spaces, tabs, and line breaks normally separate lexemes but do not affect the grammar of this language, so the scanner consumes and discards them. Their line and column effects should still be recorded for diagnostics. They would become tokens only in a whitespace-sensitive language or inside a string literal.

### (b) Lexical Recognition and Scanning

**1. Recognition of >=, &&, and >**

For >=, the scanner reads > and uses one-character lookahead. Because the next character is =, it returns one GE token for >= rather than separate GT and ASSIGN tokens. For &&, after reading the first &, it checks the next character; a second & produces one AND token. A lone & would be a lexical error unless the language defines it. For > followed by a blank, lookahead shows that = is absent, so the scanner returns the single-character GT token and leaves the blank to the whitespace rule.

Lookahead is therefore used to decide whether a character begins a longer operator. The scanner consumes the extra character only when the longer token is valid; otherwise it retracts or leaves that character for the next scan.

**2. Token stream for the first three statements**

```
Statement 1: KW_FLOAT(float) ID(basic) ASSIGN(=) FLOAT_CONST(25000.50) SEMI(;)
Statement 2: KW_INT(int) ID(bonus) ASSIGN(=) INT_CONST(5000) SEMI(;)
Statement 3: KW_IF(if) LPAREN(() ID(basic) GE(>=) INT_CONST(20000) AND(&&)
ID(bonus) GT(>) INT_CONST(0) RPAREN()) LBRACE({) ID(total) ASSIGN(=) ID(basic)
PLUS(+) ID(bonus) SEMI(;) RBRACE(})
```

A boundary is detected when the next character cannot extend the current token under any rule. Blanks end words and numbers; punctuation such as ;, (, ), {, and } is itself a token; and lookahead determines whether operator prefixes extend to two-character operators. The decimal point remains part of 25000.50 because the complete sequence matches the floating-point rule.

**3. Unrecognized characters**

The scanner should report a lexical error containing the offending character and its line and column, avoid returning it as a normal token, consume at least that character to prevent an infinite loop, and then either continue scanning for additional errors or terminate according to the compiler's recovery policy.

### (c) Longest-Match Analysis

**1. Effect of maximal munch**

At >=, both > and >= begin valid patterns, but >= is longer, so it becomes one relational token. At 25000.50, the scanner continues beyond the integer-looking prefix because digits, a decimal point, and more digits form the longer floating-point token.

**2. Integer rule placed before floating-point rule**

In a correct LEX/FLEX scanner, rule order does not cause 25000.50 to become an integer when the floating rule matches more characters; longest match is applied before rule priority. A poorly implemented first-match scanner could incorrectly produce INT(25000), an error for '.', and INT(50). The specification avoids this by using maximal munch, defining the complete floating pattern, and retaining a diagnostic rule for a stray decimal point. Placing the floating rule before the integer rule is also a clear defensive convention, especially when patterns may tie.

**3. Keyword recognition for print**

The scanner first recognizes the complete word print. It then returns KW_PRINT because the lexeme exactly matches a reserved word. The longer lexeme printer must remain an identifier because maximal munch selects printer, not the shorter prefix print. Exact keyword lookup prevents accidental prefix matching.

### (d) Extension and Evaluation

**1. Identifier beginning with underscore**

```
Revised rule: [A-Za-z_][A-Za-z0-9_]*
```

Valid examples: _basic and _payroll2. The rule still rejects an identifier beginning with a digit.

**2. Adding payroll as a keyword**

Add payroll to the reserved-word set or keyword table. The exact lexeme payroll becomes KW_PAYROLL. The inputs payrollData and payroll_total are longer lexemes matching the identifier rule, so they remain identifiers. The scanner must never recognize a keyword merely because it is a prefix of a larger identifier.

**3. Proposed tests**

Test 1 — valid declarations and decimal: float wage_1 = 1250.75;  Expected: KW_FLOAT, ID, ASSIGN, FLOAT_CONST, SEMI.

Test 2 — keyword boundary and operators: if (payrollData >= 10 && payroll_total > 0) print(payrollData); Expected: KW_IF, separators, identifiers, GE, INT_CONST, AND, GT, KW_PRINT, and SEMI; payrollData and payroll_total must not become KW_PAYROLL.

Test 3 — invalid input: int 2bonus = 5 @ 1; Expected: the scanner may tokenize 2 as an integer and bonus as an identifier, but later syntax analysis rejects the declaration; @ must immediately produce a lexical error with its position. This test checks both lexical error detection and the distinction between lexical and syntactic errors.

## Q2. Design and Evaluate a LEX Specification for a Sensor Monitoring Language

### (a) Token and Pattern Design

**1. Regular expressions**

```
Identifier: [A-Za-z][A-Za-z0-9_]*
Integer: [0-9]+
Decimal: [0-9]+\.[0-9]+
String literal: \"([^\"\\\n]|\\.)*\"
Keywords: sensor|read|display|if
Arithmetic operators: [+-/*]
Relational operators: <=|>=|==|!=|<|>
Assignment: =
Separators: [(){};,]
Comment: #[^\n]*
Whitespace: [ \t\r\n]+
```

**2. Keyword versus identifier**

The complete lexeme sensor exactly matches the keyword rule, whereas sensorValue is a longer identifier. LEX/FLEX chooses the longest match; if equal-length keyword and identifier rules both match sensor, the earlier keyword rule wins. An equivalent design is to match all names with the identifier pattern and use a reserved-word table in the action.

**3. Multi-character relational operators**

The scanner must not prematurely return < when the input is <=, or > when the input is >=. Maximal munch normally chooses the two-character token, and placing <= and >= before < and > makes the intended priority explicit and protects hand-written scanner implementations.

### (b) LEX Program Construction

The following specification includes declarations and rules. It prints token type and lexeme, ignores whitespace and comments, tracks line numbers, and diagnoses unterminated strings and other invalid characters.

```
%option noyywrap yylineno
%{
#include <stdio.h>
%}
DIGIT      [0-9]
LETTER     [A-Za-z]
ID         {LETTER}({LETTER}|{DIGIT}|_)*
INT        {DIGIT}+
DEC        {DIGIT}+\.{DIGIT}+
STR        \"([^\"\\\n]|\\.)*\"
UNTERM     \"([^\"\\\n]|\\.)*
%%
"sensor"|"read"|"display"|"if"   { printf("KEYWORD(%s)\n", yytext); }
{DEC}      { printf("DECIMAL(%s)\n", yytext); }
{INT}      { printf("INTEGER(%s)\n", yytext); }
```

```
{STR}      { printf("STRING(%s)\n", yytext); }
"<="|">="|"=="|"!="   { printf("REL_OP(%s)\n", yytext); }
"<"|">"                    { printf("REL_OP(%s)\n", yytext); }
"+"|"-"|"*"|"/"        { printf("ARITH_OP(%s)\n", yytext); }
"="                        { printf("ASSIGN(%s)\n", yytext); }
[(){};,]                   { printf("SEPARATOR(%s)\n", yytext); }
#[^\n]*                    { /* ignore comment */ }
[ \t\r\n]+                { /* ignore whitespace */ }
{ID}                       { printf("IDENTIFIER(%s)\n", yytext); }
{UNTERM}                   { fprintf(stderr, "Lexical error at line %d:
unterminated string %s\n", yylineno, yytext); }
.                          { fprintf(stderr, "Lexical error at line %d: invalid
character %s\n", yylineno, yytext); }
%%
int main(void) { yylex(); return 0; }
```

The valid string rule is placed before the unterminated-string rule. A properly closed string matches the longer valid pattern. If an opening quote is followed by characters but no closing quote before the line ends, UNTERM matches the remaining text and reports the line and offending lexeme. The final dot rule catches any other single invalid character. In a production scanner, a column counter would be added to the diagnostic as well.

### (c) Token Stream Validation

**1. Expected token sequence**

```
KEYWORD(sensor) IDENTIFIER(temperature) SEPARATOR(;)
KEYWORD(read) IDENTIFIER(temperature) SEPARATOR(;)
KEYWORD(if) SEPARATOR('(') IDENTIFIER(temperature) REL_OP(>=) INTEGER(30)
SEPARATOR(')') SEPARATOR('{')
KEYWORD(display) STRING("High temperature") SEPARATOR(';') SEPARATOR('}')
```

**2. Classification of returned lexemes**

Keywords: sensor, read, if, display. Identifiers: each occurrence of temperature. Constant: 30. Relational operator: >=. Separators: ;, (, ), {, }, and the second semicolon. String literal: "High temperature". Whitespace is ignored, and there are no comments or assignment/arithmetic operators in this sample.

**3. String as one lexical unit**

After reading the opening quotation mark, the string rule consumes every permitted character—including the embedded blank—until the matching closing quotation mark. Therefore "High temperature" is returned as one STRING token rather than two identifiers. Escaped characters are accepted by the \\. alternative, while an unescaped newline ends the valid pattern and triggers the unterminated-string diagnostic.

### (d) Testing and Evaluation

**1–2. Additional test inputs and expected results**

Decimal test: sensor humidity; humidity = 45.75; Expected: KEYWORD(sensor), IDENTIFIER(humidity), SEPARATOR(;), IDENTIFIER(humidity), ASSIGN(=), DECIMAL(45.75), SEPARATOR(;).

Multi-character operator test: if (temperature <= 18) display "Low"; Expected: KEYWORD(if), (, IDENTIFIER, REL_OP(<=), INTEGER(18), ), KEYWORD(display), STRING("Low"), ;.

Invalid-character test: read temperature @ 2; Expected: normal tokens for read and temperature, a lexical error for @ with its position, then INTEGER(2) and ; if error recovery continues.

**3. Evaluation and edge case**

These tests cover decimals, keyword/identifier recognition, assignment, a two-character relational operator, separators, strings, and invalid characters. Together with the original input they exercise most major rules, but they do not fully validate comments, all operators, keyword-prefix identifiers, or malformed literals. An

important additional edge case is: display "High temperature; followed by a newline. It must produce one unterminated-string error and must not incorrectly tokenize the words inside as identifiers. A separate useful boundary case is sensor sensorValue; to confirm exact keyword recognition.

## Q3. Trace a Source Expression from Lexical Analysis to Intermediate Code

### (a) Lexical Analysis

**1. Lexemes and classifications**

finalAmount — identifier; = — assignment operator; baseAmount — identifier; + — arithmetic operator; taxRate — identifier; * — arithmetic operator; subtotal — identifier; ; — separator/statement terminator. There is no numeric constant in this statement.

**2. Complete token stream**

```
ID(finalAmount) ASSIGN(=) ID(baseAmount) PLUS(+) ID(taxRate) MUL(*) ID(subtotal)
SEMI(;)
```

**3. Whitespace and semicolon**

Whitespace separates adjacent tokens and is discarded after positions are updated. The semicolon has no numerical value, but it is grammatically significant: it marks the end of the assignment statement, helps the parser select the correct production, and provides a synchronization point for error recovery.

### (b) Syntax and Symbol Information

**1. Syntactic structure and precedence**

```
Assignment(finalAmount, Add(baseAmount, Multiply(taxRate, subtotal)))
Equivalent grouping: finalAmount = baseAmount + (taxRate * subtotal);
```

Multiplication has higher precedence than addition, so taxRate * subtotal forms a subtree before it is combined with baseAmount. Assignment has lower precedence and binds the full right-hand expression to finalAmount.

**2. Symbol-table information**

The identifiers finalAmount, baseAmount, taxRate, and subtotal require symbol-table entries. Typical fields include name, declared type, scope, storage class, memory offset/address, size, initialization status, mutability, declaration location, and possibly current compile-time value. Semantic analysis uses these entries to check that the identifiers are declared, visible, and type-compatible.

**3. Parser use of the token stream**

The syntax analyzer matches the sequence against a production such as assignment_statement → ID ASSIGN expression SEMI. It then parses the expression using precedence-aware grammar levels, for example expression → expression + term and term → term * factor. The resulting parse tree or abstract syntax tree identifies the left-hand destination and the ordered operations on the right-hand side.

### (c) Intermediate Representation Analysis

**1. Three-address code**

```
t1 = taxRate * subtotal
finalAmount = baseAmount + t1
```

**2. Reason for multiplication first**

The intermediate representation preserves the parse tree and the language's precedence rules. Generating the multiplication instruction first ensures that baseAmount is added to the product rather than being multiplied after an incorrect addition.

**3. Temporary variable**

t1 is a compiler-generated temporary that stores the product taxRate * subtotal. It breaks a complex expression into simple three-address instructions, makes evaluation order explicit, and gives later optimization and code-generation phases a value that can be placed in a register or temporary memory location.

### (d) Phase Dependency Analysis

**1. Information passed between phases**

Lexical analysis supplies an ordered stream of token kinds, lexeme values or symbol-table references, and source positions. Syntax analysis adds grammatical structure by building a parse tree or AST. Semantic analysis then obtains and verifies declarations, types, scopes, operand compatibility, assignment compatibility, and any required implicit conversions.

**2. If * were classified as an identifier**

The parser would receive ID where it expects MUL. The token sequence around taxRate * subtotal could appear as consecutive identifiers, which violates the expression grammar. This would likely produce a syntax error near *, prevent construction of the multiplication subtree, and may trigger misleading recovery errors at subtotal or the semicolon.

**3. Propagation of recognition errors**

A wrong, missing, or split token changes the parser's input. The parser may reject a correct source statement, build the wrong tree, or recover at an incorrect location. If an incorrect tree survives, semantic checks may report false type errors and intermediate-code generation may emit operations in the wrong order, omit an operation, use the wrong operands, or stop completely. Precise lexical diagnostics therefore prevent cascading errors in later phases.

### (e) Advanced Compiler Perspective

**1. Adding exponentiation **

Add the token rule ** before or alongside the rule for *. When the first * is read, the scanner looks ahead: a second * produces one POW token; otherwise it produces MUL. Maximal munch ensures ** is selected because it is longer. The parser must separately define exponentiation precedence and associativity, commonly higher than multiplication and right-associative.

**2. Line and column positions**

Attaching a source span to every token lets later phases report messages such as “unexpected identifier at line 4, column 18” or underline the exact operand involved in a type mismatch. Positions also improve error recovery, IDE navigation, warning ranges, and the mapping of generated code back to source statements.

**3. Test statements**

```
result = base * rate + bonus;
powerTotal = base ** exponent + adjustment * factor;
```

The first statement verifies single * and + between identifiers. The second verifies that ** becomes one POW token, the later * remains MUL, + is recognized, and names containing multiple letters are returned as complete identifiers.

## Conclusion

A correct lexical analyzer depends on precise regular expressions, longest-match scanning, exact keyword handling, controlled lookahead for operator prefixes, and useful error recovery with source positions. The token stream must preserve the distinctions required by parsing and semantic analysis because an early classification error can propagate into an invalid syntax tree and incorrect intermediate code.